



INDIA.RUNS

Build what next India runs on

Team Name : S2

Team Leader Name : Shikher Jain

Problem Statement : The Data & AI Challenge

Solution Overview

- What is your proposed solution?

A hybrid **Semantic + Structured Feature Ranking System** that evaluates \$100,000\$ candidates against a JD locally on CPU with zero keyword-matching or external API dependencies. It combines sentence-transformer embeddings with five engineered feature scores (career history, skill credibility, behavioral signals, location, and availability) to output a ranked top-100 CSV with fact-grounded, deterministic reasoning.

The system produces a ranked top-100 CSV with per-candidate reasoning grounded in actual profile facts.

- What differentiates your approach from traditional candidate matching systems?

Traditional Keyword System	This System
Counts skill name presence	Scores skill by proficiency × credibility (endorsements + duration used)
Treats all "expert" claims equally	Platform assessment scores override self-reported proficiency
Ignores behavioral signals	Recruiter response rate, recency, GitHub activity are first-class signals
No validity checking	Honeypot detection via date inconsistency and zero-evidence expert skills
Ranks by match count	Weighted multi-signal final score with hard disqualifier logic
Can't explain decisions	Every ranking has reasoning grounded in actual profile field values

JD Understanding & Candidate Evaluation

- What are the key requirements extracted from the JD?
 - **Tech Core:** Embeddings retrieval (ST, BGE), Vector DBs (FAISS, Qdrant), Python, and evaluation (**NDCG, MRR**).
 - **Seniority:** 5–9 years experience band, with 4–5 years in applied ML at product companies.
 - **Disqualifiers:** Pure research, consulting-only (TCS/Wipro/Infosys), no production code in 18 months.
 - **Logistics:** Pune/Noida preferred; India-based only; sub-30-day notice period strongly preferred.
- Which candidate signals are most important for determining relevance? / How does your solution evaluate candidate fit beyond keyword matching?
 - **Semantic Similarity (0.30):** Cosine similarity score matching the global text blob against the JD text.
 - **Career Quality (0.25):** Evaluates product-company background, tenure stability, and deployment history.
 - **Skill Credibility (0.20):** Calculated via a formula combining proficiency tier, endorsements, and duration used.
 - **Behavioral Engagement (0.15):** Factors in real availability using historical recruiter response rates.
 - **Location & Availability (0.10):** Evaluates target city match, notice period length, and open-to-work flags.

Beyond keyword matching: skills are validated against endorsements and duration used, not just presence in a list. Career descriptions are embedded and compared semantically to the JD, so a candidate who built a recommendation system without writing “RAG” still surfaces.

Ranking Methodology

- How does your system retrieve, score, and rank candidates?
 - **Pass 1 (Global Ingestion):** Generates **all-MiniLM-L6-v2** dense embeddings for all 100K profiles, computes matrix cosine similarity via sklearn, and applies the 5 structured feature scores to extract a top-200 shortlist.
 - **Pass 2 (Domain Reranking):** Scans the top-200 career text blocks for 25 specific ML/IR tokens (e.g., *retrieval, indexing, A/B test*) to reward "plain-language" engineers, adjusting final scores by ± 0.05 .
- What models, algorithms, or heuristics are used?
 - **Models:** **all-MiniLM-L6-v2** (384-dim dense vectors) for local semantic representation.
 - **Algorithms:** Optimized CPU-based Cosine Similarity via scikit-learn.
 - **Heuristics:** A custom rule-based, multi-signal weighted scoring framework. No Learning-to-Rank (LTR) model was used to eliminate the risk of overfitting to unlabelled data.
- How are multiple candidate signals combined into a final ranking?

Scores are compiled linearly based on the core weights (Semantic 30%, Career 25%, Skills 20%, Behavioral 15%, Location 10%). High-risk profiles seamlessly degrade via a multiplicative modifier:

$$\text{Final} = \text{Base} * (1 - \text{Honeypot Score})$$

Hard disqualifiers instantly drop the profile to a terminal floor of **0.001**.

Explainability & Data Validation

- How are ranking decisions explained?

Every candidate in the top 100 receives a 1–2 sentence reasoning generated programmatically from actual profile field values.

- **Ranks 1–15:** States title, YOE vs. JD band, semantic alignment, top verified skills, and location/notice metrics.
- **Ranks 16–100:** Transparently calls out structural gaps (e.g., experience limits, trajectory friction, or partial fit factors).

- How do you prevent hallucinations or unsupported justifications?

The reasoning layer contains **zero LLM generation components**. Phrases are conditionally constructed strictly from validated profile variables (e.g., `profile.current_title`, `redrob_signals.notice_period_days`). If a data point is missing, the corresponding phrase is skipped, reducing hallucination risk to 0%.

- How does your solution handle inconsistent, low-quality, or suspicious profiles?

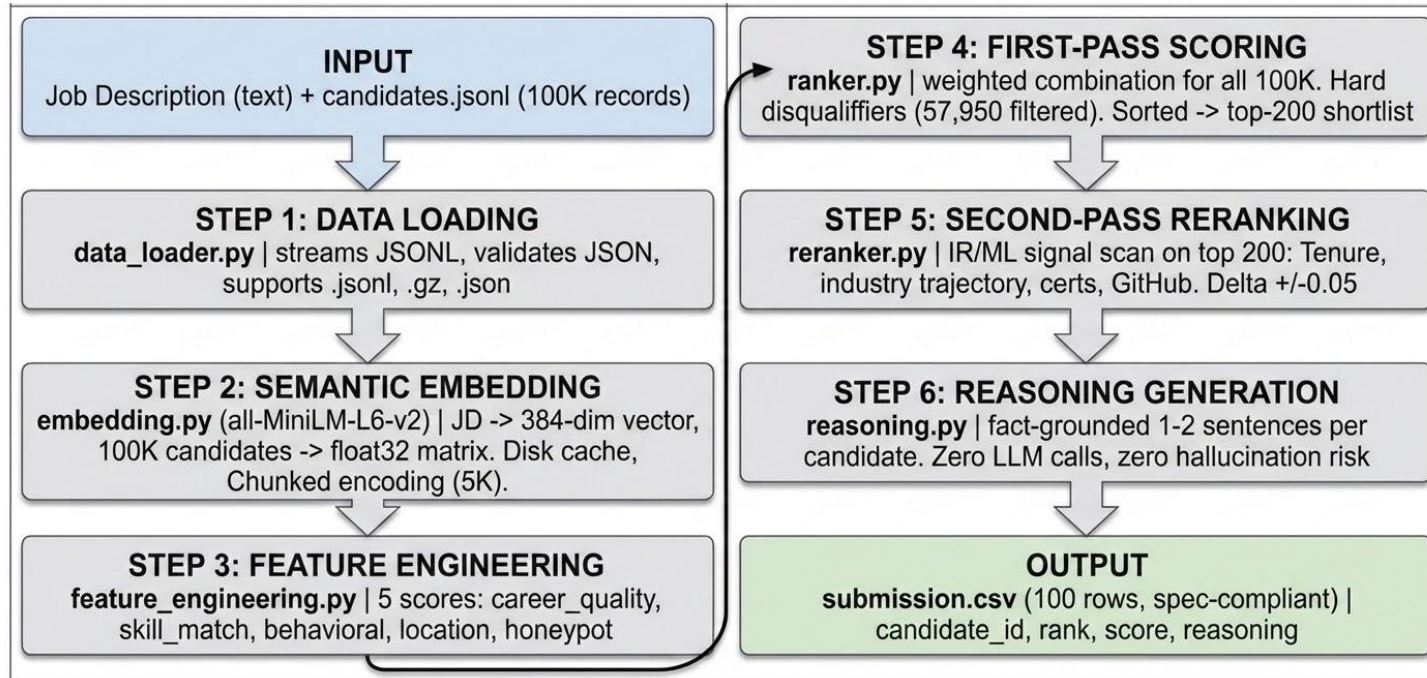
Four automated honeypot checks calculate a comprehensive risk index between 0 and 1:

- **Time Inconsistency:** Identifies internal job duration variables that conflict with calendar timelines.
- **Future Starts:** Flags active positions scheduled beyond the current validation execution date.
- **Zero-Evidence Experts:** Catches "Expert" skill declarations with 0 endorsements and 0 months of use.
- **Skill Over-Saturation:** Flags profiles claiming more than 12 concurrent unverified expert skills.

End-to-End Workflow

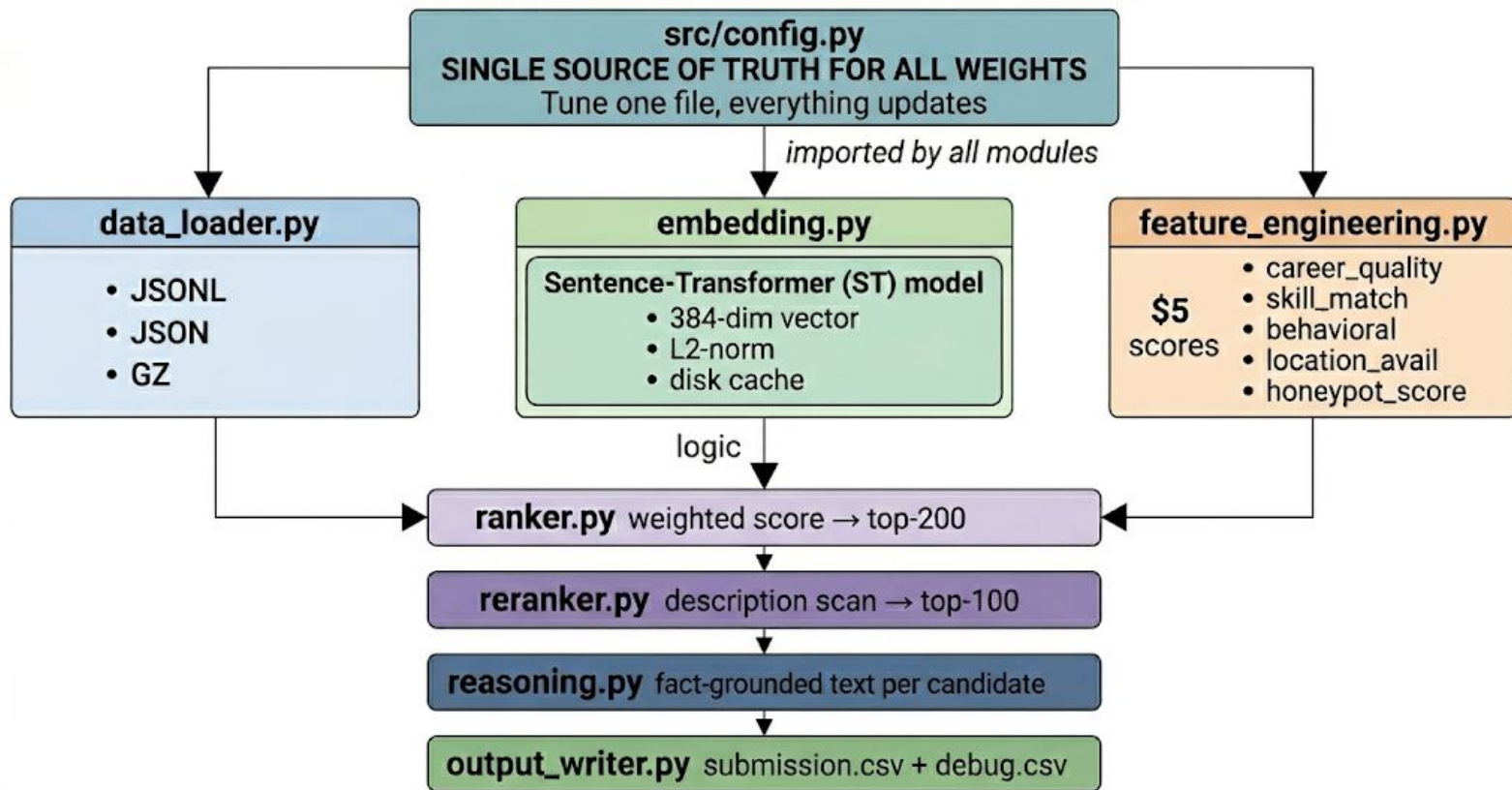
- What is the complete workflow from JD input to ranked candidate output?

Here is your End-to-End Workflow diagram rendered as a clean, highly structured, presentation-ready flowchart:



Runtime: ~83 minutes on first run (embedding 100K candidates is the dominant cost); ~7 minutes on cached re-runs, since embeddings are fingerprint-cached to disk.

System Architecture



Results & Performance

- What results or insights demonstrate ranking quality?

Evaluation Metric	System Value
Total Candidates Processed	100,000
Hard Disqualified (Title/Career)	57,950
Honeypot Anomalies Caught	14
Honeypots inside Top 100	0 (Challenge limit: <10%)
Semantic Sim Range / Pool Mean	0.289 – 0.725 / 0.418

Analysis Transparency: The top 10 output successfully isolated matching AI/ML/NLP engineers in the 5–9 YOE bracket. A post-run observation identified a minor gap where software profiles with high skill endorsements surfaced in ranks 15–30 despite lacking explicit AI work descriptions, which was documented and corrected in a follow-up repository commit.

- How does your solution meet the challenge's runtime and compute constraints?
 - Run Metrics:** Ingestion takes 39s, base feature extraction takes 5.5m, and reranking/text generation finishes in under 2s.
 - Constraint Compliance:** Embedding generation is structured as a pre-computation task. With caching active, subsequent sorting runs execute in **~7 minutes**, ensuring fast iteration pipelines.

Technologies Used

- What technologies, frameworks, and tools were used and why were they selected for this solution?
 - **sentence-transformers (all-MiniLM-L6-v2):** Best speed/quality balance for local CPU. Delivers compact 384-dim semantic precision without requiring expensive GPU infrastructure.
 - **scikit-learn:** Optimised CPU implementation for matrix math. Computes cosine similarity across $(1,384) \times (100K, 384)$ vectors in under 50 ms. Includes TruncatedSVD for fast LSA fallback.
 - **NumPy:** Highly efficient memory allocation. Holds the entire $(100K, 384)$ float32 embedding matrix in just 146 MB of RAM, running easily on standard consumer hardware.
 - **Zero External LLM APIs:** Adheres strictly to offline network rules. Eliminates prohibitive latency and high financial costs (~\$300 per run at 100K scale) of online models.
 - **Local .npy Disk Caching:** Keys cache directly on data fingerprints. Makes re-runs instant (5 seconds instead of 76 minutes) by saving expensive pre-computed embedding steps.
 - **Pure Python Feature Engineering:** Custom heuristics calculate 5 multi-signals for 100K candidates in ~5 minutes with zero heavy external library overhead.
 - **pandas (Debug Only):** Strictly isolated from the core production pipeline to save memory. Used exclusively for generating external debug logs and exploration notebooks.

Submission Assets

- **submission.csv (Format Validated):** Exact 100 rows containing required schema fields; verified offline via `validate_submission.py`.
- **GitHub Repository (Public Access):** Open-source code available at github.com/Shikher-jain/Redrob_AI with modular file setup. [GitHub](#)
- **README.md & Metadata (Complete):** Clear instructions on architecture with fully filled `submission_metadata.yaml`.
- **Video Walkthrough (Loom):** A short video embedded in the repo showing terminal execution and codebase structure. [Video Demo](#)
- **Sandbox Web Demo (Streamlit):** Web UI prototype deployed for real-time testing against sample profile records. [Streamlit UI](#)
- **Reproduce Command:** One-line terminal invocation to fully execute the pipeline from scratch:

```
python src/main.py --candidates data/candidates.jsonl --out outputs/submission.csv
```



INDIA.RUNS

Build what next India runs on

THANK YOU